

```

sound/pci/cs46xx/imgs/cwcbnhack.h \
sound/pci/cs46xx/imgs/cwcdma.asp \
:do
if test ! $f; then
die $f is not present, something is amiss
fi
done

```

For your reference, here are the functions performed by the scripts:

- **deblob-main**: The main script used to clean up the Linux tarball.
 - **deblob-check**: The script that finds blobs. It may also clean up work.
 - **deblob-2.6.#:** The script that cleans up blobs from within a given exploded Linux source tree.
- Now, coming to the removal:

```
# Identify the tarball.
```

```
sed -i 's,"EXTRAVERSION:",&lib$extra," Makefile
```

```
#####
```

```
# Removed ATM Drivers #
```

```
#####
```

```
# ATM_AMBASSADOR - Madge Ambassador (Collage PCI 155
Server)
```

```
clean_blob drivers/atm/atmsar11.data
```

```
# ATM_FORE200E_PCA
```

```
# ATM_FORE200E_SBA - SBA-200E
```

```
clean_kconfig drivers/atm/Kconfig 'ATM_FORE.*'
```

```
clean_mk CONFIG_ATM_FORE200E drivers/atm/Makefile
```

```
clean_file drivers/atm/pca200e.data
```

```
clean_file drivers/atm/pca200e_ecc.data
```

```
clean_file drivers/atm/sba200e_ecc.data
```

```
clean_kconfig drivers/atm/Kconfig 'ATM_AMBASSADOR'
```

```
clean_mk CONFIG_ATM_AMBASSADOR drivers/atm/Makefile
```

The interesting point is that maintaining Linux-libre is not a time-consuming process. And there are scripts that will inform the project manager whether there is anything that needs manual intervention.

David Woodhouse suggested having a separate branch of the kernel source tree (which would be excluded from a normal kernel build process) for non-free firmware. Thus, the non-free firmware could be distributed in a separate package. But the idea of 'complete freedom', as proposed by Linux-libre, is not respected here.

Outline of Linux kernel

Now let's consider the idea of tasks. We have already seen that Linux supports multi-tasking. Any application that runs the memory of the system and

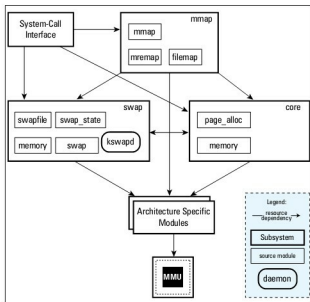


Figure 4: Memory manager structure

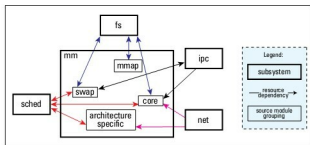


Figure 5: Memory manager dependencies

shares the system's resources may be termed as a task. And by multi-tasking, we are actually referring to the effective sharing of these resources among the tasks. Here, the system can switch from one task to another after a given *timeslice* time (say 10 ms). This gives an impression that many tasks are handled simultaneously.

Here are the detailed steps of the process: Let's say *task1* is running and using the resources. Then, a resource request will be made that forces the system to put the *task1* in the *block list* and choose *task2* from the *ready list* for task switching. This is what happens when it comes to two tasks. You can extend this idea to N number of tasks by choosing a timer IRQ for the switching stage.

Having discussed these ideas, we can now go back to the sub-system structure of the operating system.

The process scheduler is employed to:

- Allow processes to create fresh copies
- Send signals to the user processes
- Manage the timer
- Select the process that can access the CPU
- Receive interrupts and route them to the